

EN3551 — Assignment 1
Detecting Harmonics in Noisy Data and Signal Interpolation using DFT

Name: D J de Mel
Index Number: 230121D

September 3, 2026

Contents

1	Introduction	2
2	Harmonic Detection	2
2.1	Signal Description	2
2.2	DFT of Subsets S_1 – S_5	2
2.3	DFT Averaging Method	5
2.4	Minimum Number of Partitions L	6
2.5	Alternative Values of K	8
3	Frequency-Domain Interpolation	8
3.1	Test Signal	8
3.2	Interpolation of x_2 ($K = 1$)	8
3.3	Interpolation of x_3 ($K = 2$)	8
3.4	Interpolation of x_4 ($K = 3$)	9
3.5	Comparison and Discussion	10
A	MATLAB Code	10
A.1	Harmonic Detection	10
A.2	Interpolation	12

1 Introduction

This report presents the solutions for Assignment 1 of EN3551, which explores two applications of the Discrete Fourier Transform (DFT): (i) detecting sinusoidal harmonics buried in noisy data using DFT averaging, and (ii) interpolating a discrete-time signal in the frequency domain using zero insertion. All computations and plots were generated using MATLAB.

2 Harmonic Detection

2.1 Signal Description

The noise-corrupted test signal `xn_test` corresponding to index number 230121 was used, consisting of 1793 samples sampled at $f_s = 128$ Hz over a duration of 14 s. The signal contains three unknown harmonics, each not exceeding 64 Hz, embedded in severe additive white Gaussian noise.

2.2 DFT of Subsets S_1 – S_5

The signal was partitioned into five subsets of increasing length: S_1 (128 samples), S_2 (256 samples), S_3 (512 samples), S_4 (1024 samples), and S_5 (1792 samples). The DFT magnitude of each subset is shown below.

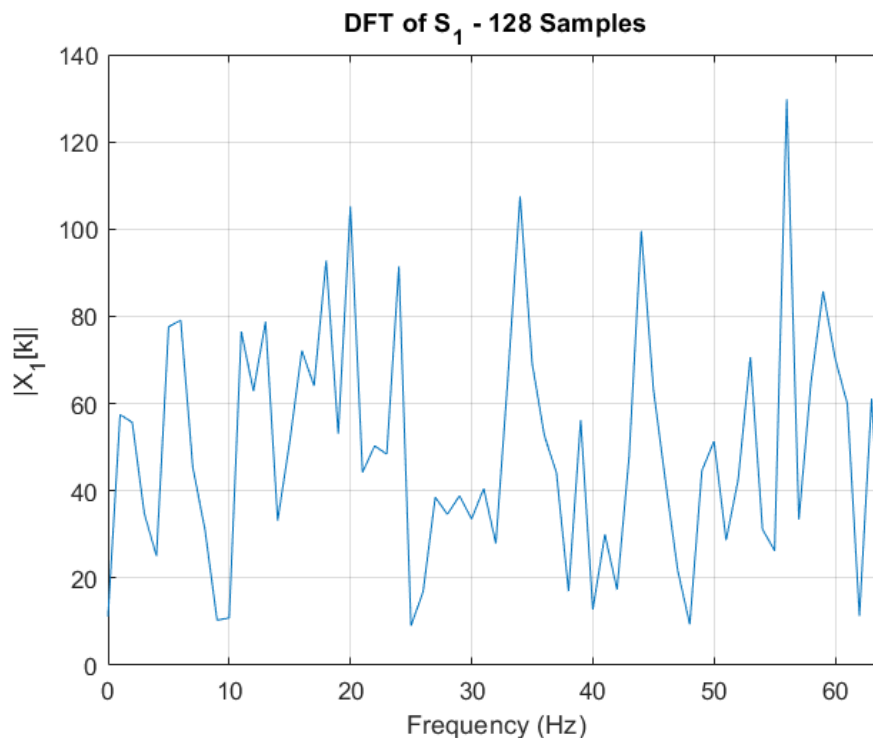


Figure 1: DFT magnitude of subset S_1 (128 samples)

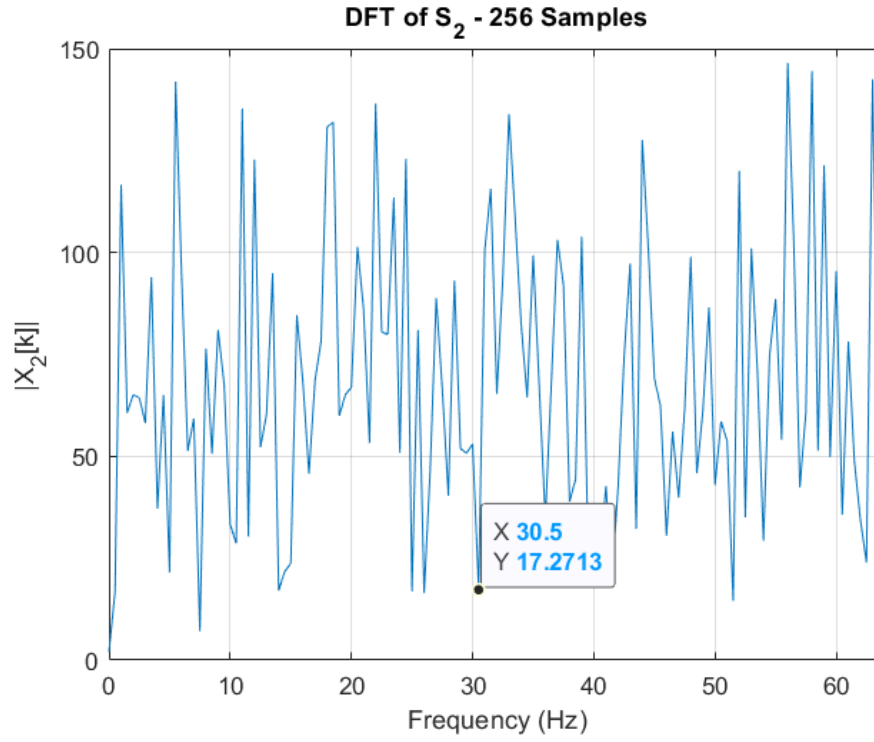


Figure 2: DFT magnitude of subset S_2 (256 samples)

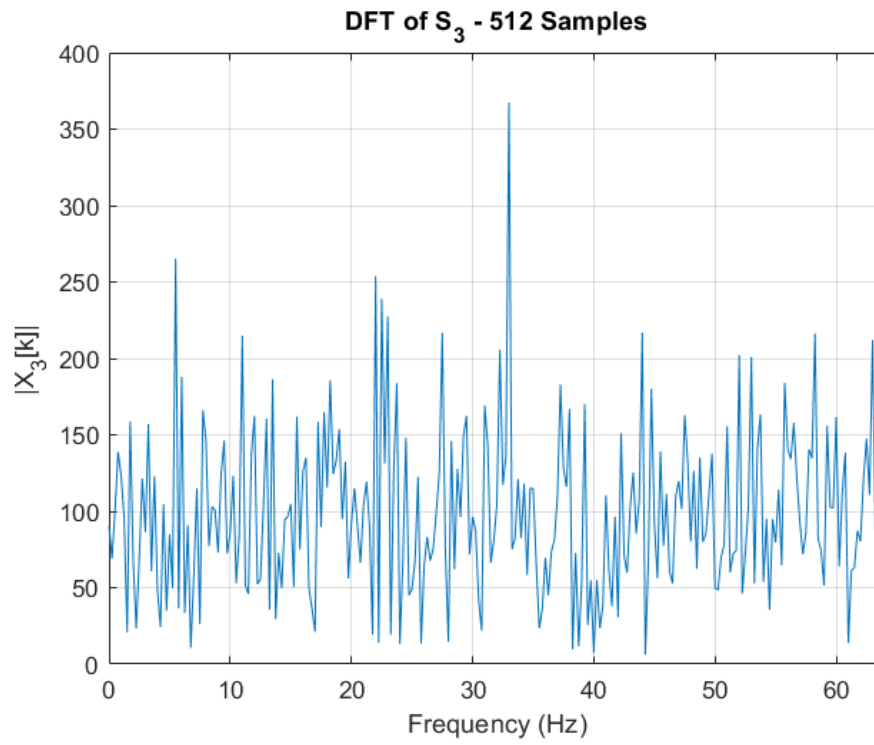


Figure 3: DFT magnitude of subset S_3 (512 samples)

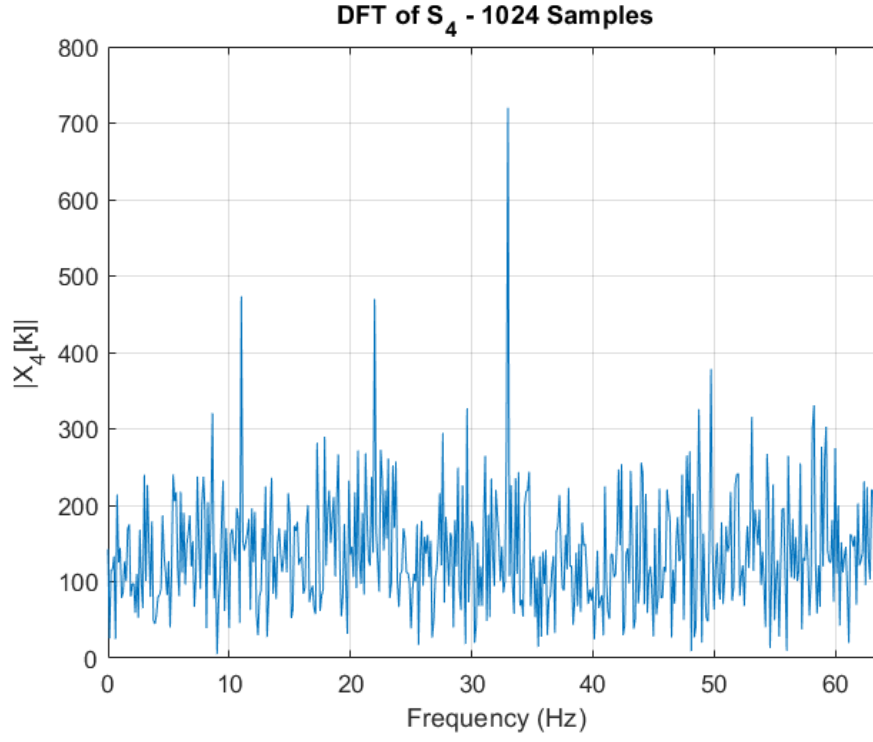


Figure 4: DFT magnitude of subset S_4 (1024 samples)

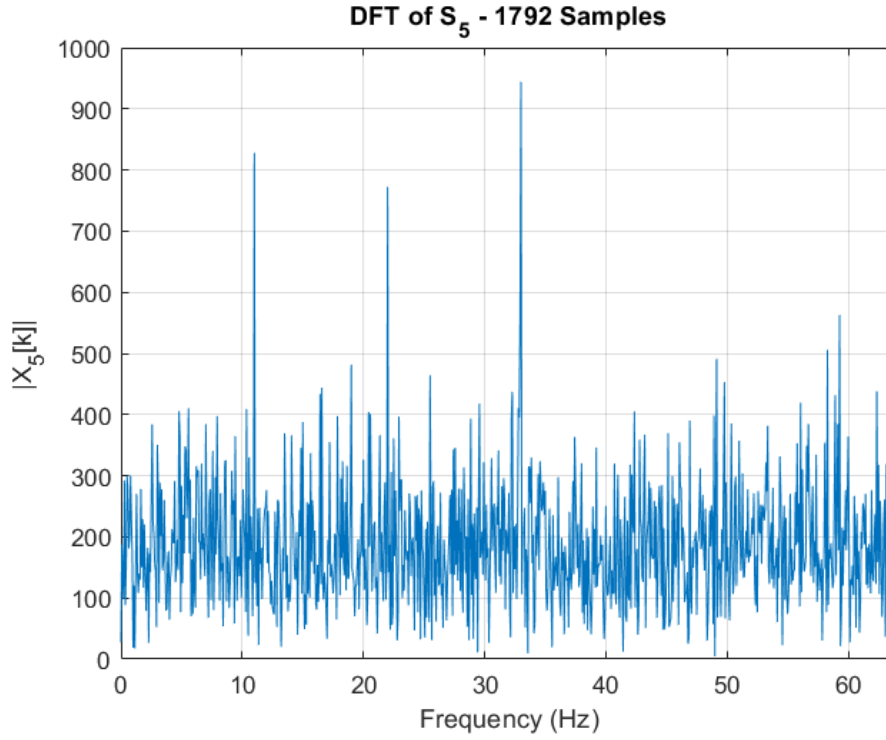


Figure 5: DFT magnitude of subset S_5 (1792 samples)

Observations. As the length of the subset increases, the 3 frequencies become more distinct and noticeable.

as N grows (S3, S4, S5), the frequency resolution ($\frac{f_s}{Length}$) improves so peaks get narrower. The observed frequencies are $f_1 = 11Hz$, $f_2 = 22Hz$, $f_3 = 33Hz$.

2.3 DFT Averaging Method

The DFT averaging method described in Section 2.2 of the assignment sheet was applied with subset length $K = 128$ and number of subsets $L = 14$ (so that $N = LK = 1792$).

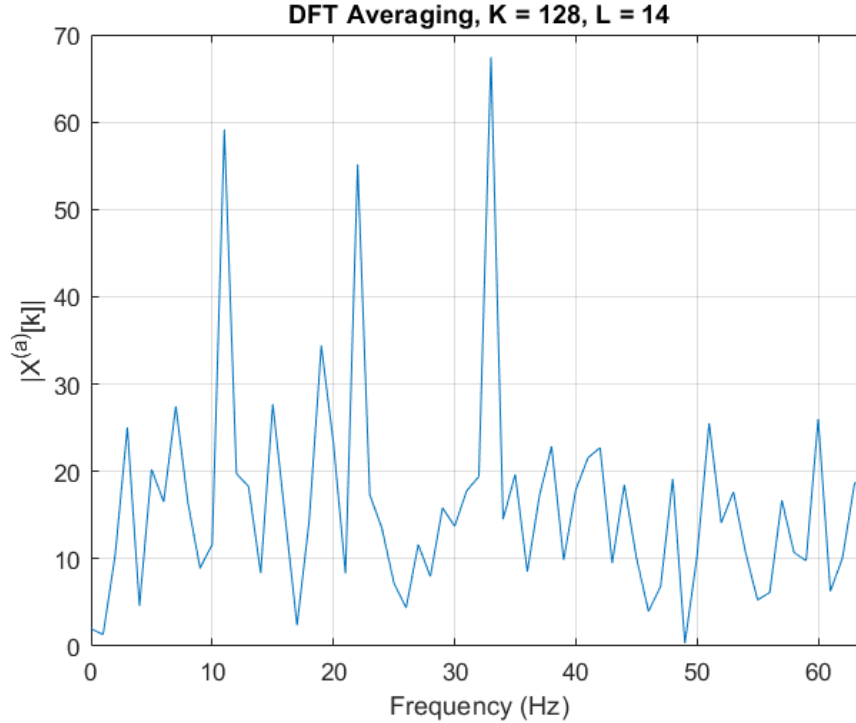


Figure 6: Averaged DFT magnitude, $K = 128$, $L = 14$

Harmonic	Frequency (Hz)
f_1	$11Hz$
f_2	$22Hz$
f_3	$33Hz$

Table 1: Identified harmonic frequencies from DFT averaging

Identified Harmonic Frequencies.

Discussion. With the noisy signal, the DFT magnitude contains a significant noise floor, making the three frequencies difficult to identify.

By dividing the 1792 samples into 14 subsets and averaging their DFTs, the random noise components tend to cancel while the frequency components corresponding to the harmonics are retained.

2.4 Minimum Number of Partitions L

To determine the smallest value of L for which the three harmonic peaks remain clearly visible, DFT averaging was repeated for several values of L (with $K = N/L$).

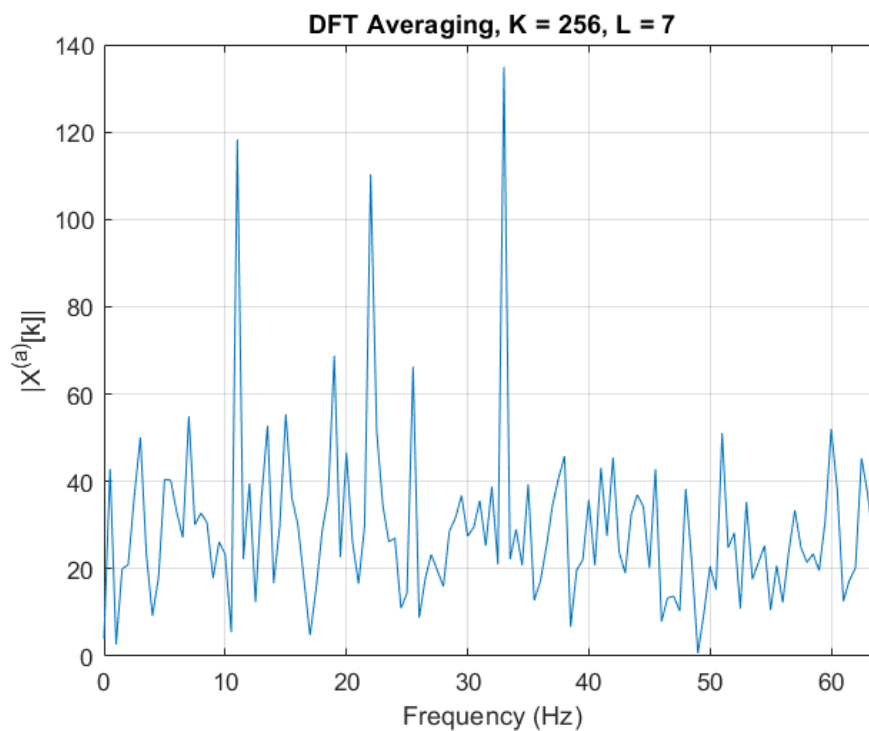


Figure 7: DFT magnitude for $L = 7$

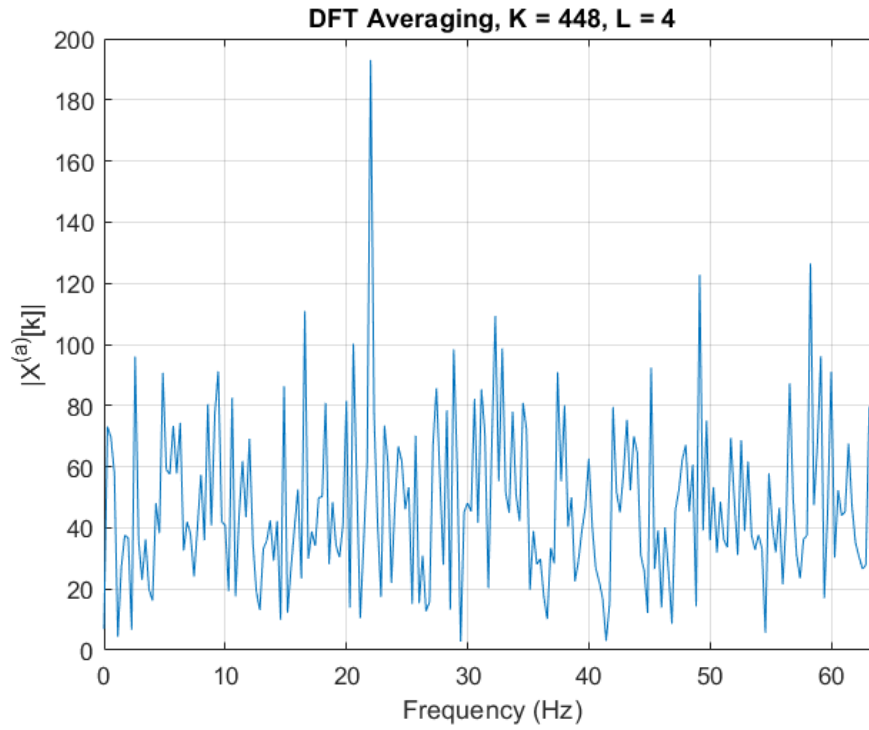


Figure 8: DFT magnitude for $L = 4$

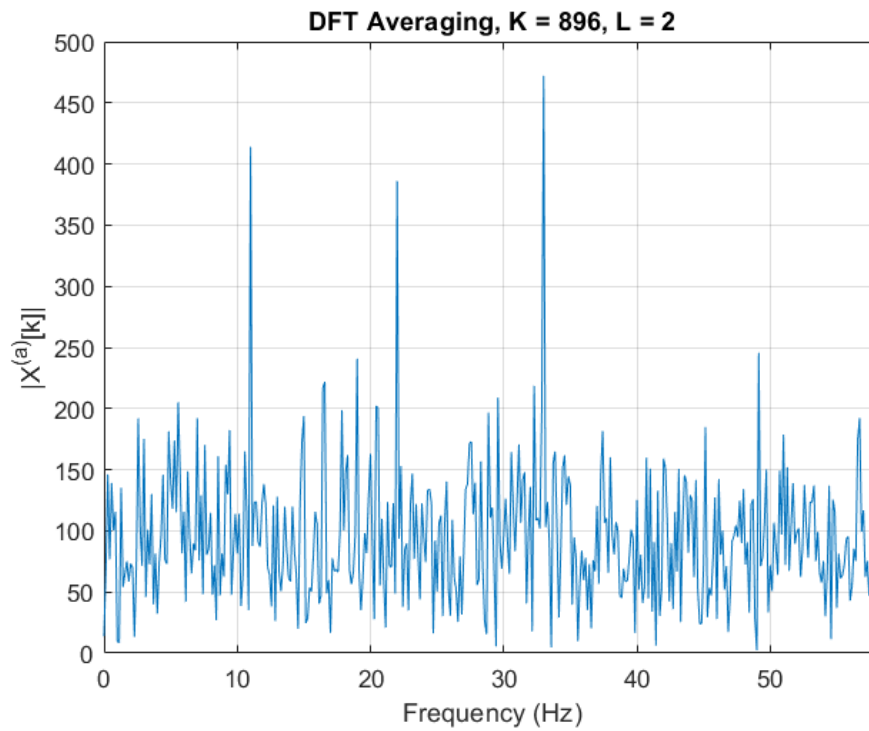


Figure 9: DFT magnitude for $L = 2$

Result. The smallest value of L for which the three peaks remain clearly distinguishable from the noise floor is $L = 2$.

When going from $L = 2$ to $L = 14$, the trend is not strictly monotonic. There's a trade-off between resolution (favoring small L) and noise suppression (favoring large L). The peaks aren't discernible at $L=4$ but they become sharper when $L=2$.

2.5 Alternative Values of K

$N = L \cdot K$ must hold exactly, with L and K both integers, since we are partitioning N samples into L equal non-overlapping chunks. If K doesn't divide N evenly, we can't split cleanly into equal segment.

3 Frequency-Domain Interpolation

3.1 Test Signal

The test signal used is the first $N = 20,000$ samples of the “Hallelujah” music clip by Handel, loaded in MATLAB using `load handel`. From this signal $x[n]$, three downsampled signals were generated: $x_2 = x(1 : 2 : N)$, $x_3 = x(1 : 3 : N)$, and $x_4 = x(1 : 4 : N)$.

3.2 Interpolation of x_2 ($K = 1$)

The signal x_2 has length 10,000, which is even. The DFT-based interpolation method (Section 2.3) was applied with $K = 1$.

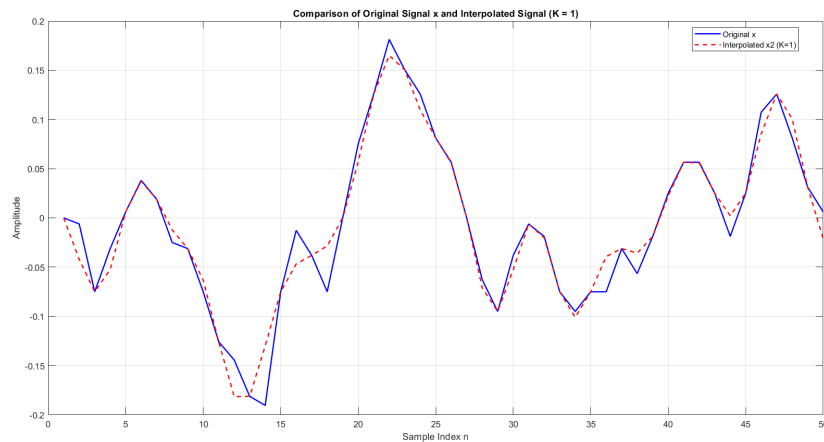


Figure 10: Comparison of the first 50 samples of the interpolated signal (from x_2) and the original signal x

2-Norm Error. $\|x_{\text{interp}} - x\|_2 = 6.1447$

3.3 Interpolation of x_3 ($K = 2$)

The signal x_3 has length 6667, which is odd. The interpolation method was applied with $K = 2$.

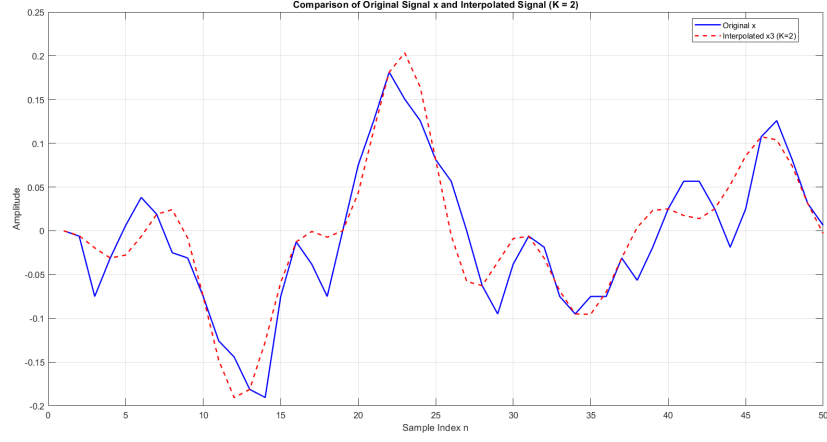


Figure 11: Comparison of the first 50 samples of the interpolated signal (from x_3) and the original signal x

2-Norm Error. $\|x_{\text{interp}} - x\|_2 = 8.3652$

3.4 Interpolation of x_4 ($K = 3$)

The signal x_4 has length 5000, which is Even. The interpolation method was applied with $K = 3$.

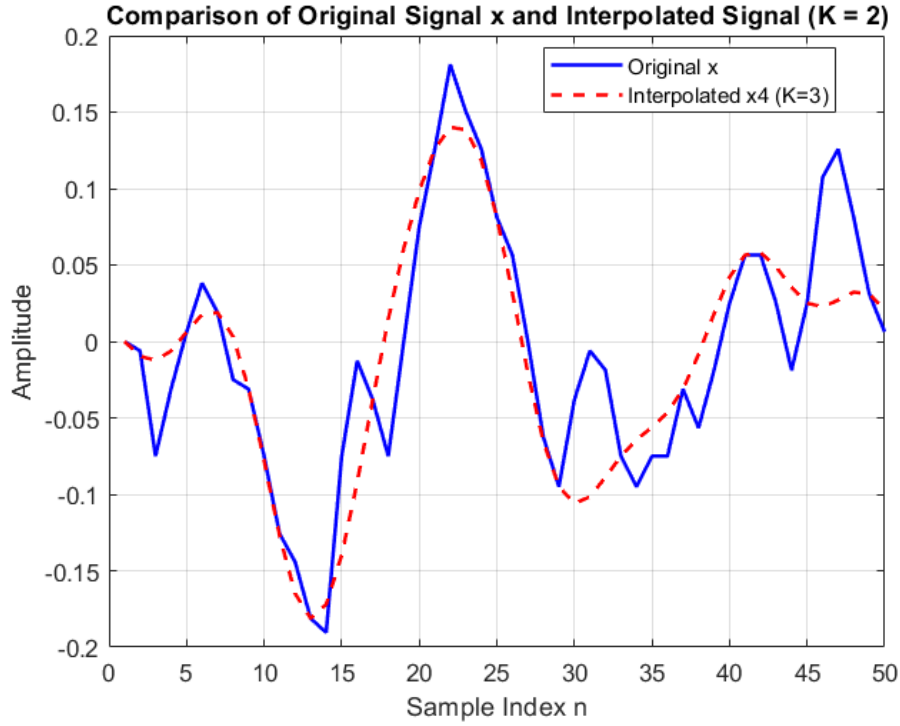


Figure 12: Comparison of the first 50 samples of the interpolated signal (from x_4) and the original signal x

2-Norm Error. $\|x_{\text{interp}} - x\|_2 = 23.4998$

3.5 Comparison and Discussion

Signal	K	Length (odd/even)	2-norm error
x_2	1	Even	6.1447
x_3	2	Odd	8.3632
x_4	3	Even	23.4998

Table 2: Summary of interpolation results

Observations and Comments. The most accurate representation of the signal is obtained when more samples have been used for the interpolation. This is expected because when we throw way more samples, reconstructing the high frequencies components become harder.

References

1. E.O. Brigham, *The Fast Fourier Transform and its applications*, Prentice Hall, 1988.

Appendix

A MATLAB Code

A.1 Harmonic Detection

```

1 data = load('signal121.mat');
2 fs = 128;
3
4 xn = data.xn_test;
5
6 S1 = xn(1:128);
7 S2 = xn(1:256);
8 S3 = xn(1:512);
9 S4 = xn(1:1024);
10 S5 = xn(1:1792);
11
12 % Number of samples
13 N1 = length(S1);
14 N2 = length(S2);
15 N3 = length(S3);
16 N4 = length(S4);
17 N5 = length(S5);
18
19 % Frequency axes with resolution fs/(No. of samples)
20 f1 = (0:N1-1)*fs/N1;
21 f2 = (0:N2-1)*fs/N2;
22 f3 = (0:N3-1)*fs/N3;
23 f4 = (0:N4-1)*fs/N4;
```

```

24 f5 = (0:N5-1)*fs/N5;
25
26 % Plot S1
27 figure;
28 plot(f1(1:N1/2+1), abs(X1(1:N1/2+1)));
29 title('DFT of S_1 - 128 Samples');
30 xlabel('Frequency (Hz)');
31 ylabel('|X_1[k]|');
32 xlim([0 64]);
33 grid on;
34
35 % Plot S2
36 figure;
37 plot(f2(1:N2/2+1), abs(X2(1:N2/2+1)));
38 title('DFT of S_2 - 256 Samples');
39 xlabel('Frequency (Hz)');
40 ylabel('|X_2[k]|');
41 xlim([0 64]);
42 grid on;
43
44 % Plot S3
45 figure;
46 plot(f3(1:N3/2+1), abs(X3(1:N3/2+1)));
47 title('DFT of S_3 - 512 Samples');
48 xlabel('Frequency (Hz)');
49 ylabel('|X_3[k]|');
50 xlim([0 64]);
51 grid on;
52
53 % Plot S4
54 figure;
55 plot(f4(1:N4/2+1), abs(X4(1:N4/2+1)));
56 title('DFT of S_4 - 1024 Samples');
57 xlabel('Frequency (Hz)');
58 ylabel('|X_4[k]|');
59 xlim([0 64]);
60 grid on;
61
62 % Plot S5
63 figure;
64 plot(f5(1:N5/2+1), abs(X5(1:N5/2+1)));
65 title('DFT of S_5 - 1792 Samples');
66 xlabel('Frequency (Hz)');
67 ylabel('|X_5[k]|');
68 xlim([0 64]);
69 grid on;
70
71 Now apply the DFT averaging method described in the Section 2 above, where
    the length of each subset is taken to be K = 128, and the number of
    subsets is taken to be L = 14.
72
73 K = 128;
74 L = 14;
75

```

```

76 P = zeros(K,L);
77
78 for n = 1:L
79     sub = S5((n-1)*K+1 : K*n);
80     DFT = fft(sub);
81     P(:,n) = DFT;
82 end
83
84 avg_dft = mean(P,2);
85
86 % Magnitude of averaged DFT
87 avg_mag = abs(avg_dft);
88
89 f = (0:K-1)*fs/K;
90
91 figure;
92 plot(f(1:K/2+1), avg_mag(1:K/2+1));
93 title('DFT Averaging, K = 128, L = 14');
94 xlabel('Frequency (Hz)');
95 ylabel('|X^{(a)}[k]|');
96 xlim([0 64]);
97 grid on;
98
99 K = 256;
100 L = 7;
101
102 P = zeros(K,L);
103
104 for n = 1:L
105     sub = S5((n-1)*K+1 : K*n);
106     DFT = fft(sub);
107     P(:,n) = DFT;
108 end
109
110 avg_dft = mean(P,2);
111
112 % Magnitude of averaged DFT
113 avg_mag = abs(avg_dft);
114
115 f = (0:K-1)*fs/K;
116
117 figure;
118 plot(f(1:K/2+1), avg_mag(1:K/2+1));
119 title('DFT Averaging, K = 256, L = 7');
120 xlabel('Frequency (Hz)');
121 ylabel('|X^{(a)}[k]|');
122 xlim([0 64]);
123 grid on;

```

A.2 Interpolation

```

1
2 function y = zero_pad(X,K)

```

```

3     N=length(X);
4     if (mod(N,2)==1)
5         N1 = (N+1)/2;
6         y = [X(1:N1) ; zeros(K*N,1) ; X(N1+1:N)];
7
8     else
9         N2 = N/2;
10        y = [X(1 : N2); X(N2+1)/2; zeros(K*N-1,1); X(N2+1)/2; X((N2+2) : N
11            )];
12    end
13    N=20000;
14    x=y(1:N);
15    %Downsampling = reducing the number of samples.
16    %Downsampling doesn't necessarily make the
17    %original waveform slower or change its physical frequency.
18    %Instead, you are representing it with fewer samples.
19
20
21    x2 = x(1 : 2 : N); %start : step : end
22    X2 = fft(x2);
23    k2=1;
24    X2_pad = zero_pad(X2,k2);
25
26    X2_inv = ifft(X2_pad);
27    X2_inv = X2_inv * (k2+1);
28    x2_interpolated = X2_inv(1:(k2+1)*(length(x2)-1)+1);
29
30    error_x2 = norm(x(1:19999)-x2_interpolated(1:19999));
31    plot(x(1:50), 'b-', 'LineWidth', 1.5);
32    hold on
33    plot(x2_interpolated(1:50), 'r--', 'LineWidth', 1.5);
34    hold off
35
36    title('Comparison of Original Signal x and Interpolated Signal (K = 1)');
37    xlabel('Sample Index n');
38    ylabel('Amplitude');
39    legend('Original x', 'Interpolated x2 (K=1)', 'Location', 'best');
40    grid on;
41
42    x3 = x(1 : 3 : N);
43    X3 = fft(x3);
44    k3=2;
45    X3_pad = zero_pad(X3,k3);
46
47    X3_inv = ifft(X3_pad);
48    X3_inv = X3_inv * (k3+1);
49    x3_interpolated = X3_inv(1:(k3+1)*(length(x3)-1)+1);
50    error_x3 = norm(x(1:19999)-x3_interpolated(1:19999));
51
52    plot(x(1:50), 'b-', 'LineWidth', 1.5);
53    hold on
54    plot(x3_interpolated(1:50), 'r--', 'LineWidth', 1.5);
55    hold off

```

```

56
57 title('Comparison of Original Signal x and Interpolated Signal (K = 2)');
58 xlabel('Sample Index n');
59 ylabel('Amplitude');
60 legend('Original x', 'Interpolated x3 (K=2)', 'Location', 'best');
61 grid on;
62
63 x4 = x(1 : 4 : N);
64 X4 = fft(x4);
65 k4=3;
66 X4_pad = zero_pad(X4,k4);
67
68 X4_inv = ifft(X4_pad);
69 X4_inv = X4_inv * (k4+1);
70 x4_interpolated = X4_inv(1:(k4+1)*(length(x4)-1)+1);
71 error_x4 = norm(x(1:19997)-x4_interpolated(1:19997));
72
73 plot(x(1:50), 'b-', 'LineWidth', 1.5);
74 hold on
75 plot(x4_interpolated(1:50), 'r--', 'LineWidth', 1.5);
76 hold off
77
78 title('Comparison of Original Signal x and Interpolated Signal (K = 2)');
79 xlabel('Sample Index n');
80 ylabel('Amplitude');
81 legend('Original x', 'Interpolated x4 (K=3)', 'Location', 'best');
82 grid on;

```